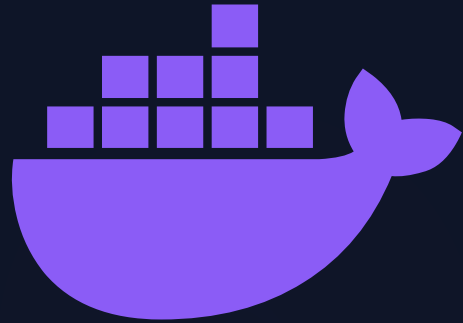




Docker

O Problema: "Na minha máquina funciona!"

O Caos

Você faz um código incrível, mas no PC do seu amigo dá erro.

Falta o Node.js 20, o banco de dados é diferente ou as configurações estão erradas.

O ambiente é **imprevisível**.

A Solução

O Docker cria uma **"caixa mágica"**.

Tudo o que seu código precisa para rodar está l
empacotado.

Se funciona na sua máquina, vai funcionar em **o
lugar**.

Docker resolve: Versões diferentes, bibliotecas faltando e configurações erradas.

Imagem vs Container (Receita vs Bolo)

TERMO	O QUE É?	ANALOGIA
Imagem	O arquivo parado, com as instruções.	📖 A Receita do Bolo
Container	A imagem "viva", rodando agora.	🍰 O Bolo pronto

💡 **Dica:** Você pode fazer 10 bolos (containers) usando a mesma receita (imagem).

Dockerfile: O Manual de Instruções

```
FROM node:20-slim
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "app.js"]
```

- 1 **FROM:** Qual "fogão" vamos usar? Aqui escolhemos o Node.js pronto para uso.
- 2 **WORKDIR:** Em qual bancada vamos trabalhar? Criamos uma pasta para o projeto.
- 3 **COPY:** Coloque os ingredientes na bancada. Copiamos seu código para o Docker.
- 4 **RUN:** Misture tudo! Instalamos as bibliotecas que seu código precisa.
- 5 **CMD:** Ligue o forno! É o comando que inicia sua aplicação de verdade.

Comandos: Build e Run (Criar e Usar)

Passo 1: Build (Criar a Receita)

```
docker build -t meu-app .
```

 "Ei Docker, use esse arquivo e crie uma imagem chamada 'meu-app' usando os arquivos desta pasta."

Passo 2: Run (Fazer o Bolo)

```
docker run -p 3000:3000 meu-app
```

 "Ei Docker, ligue o container e conecte a porta 3000 do meu PC com a dele."

Entendendo as Portas (-p)



A Analogia do Prédio

Imagine que o Container é um prédio com várias janelas (portas). Para alguém de fora conseguir entrar ou mandar uma mensagem, você precisa dizer qual janela do seu PC se conecta com a janela do prédio.

O Comando Mágico

```
-p 8080:3000
```



Janela do seu PC



Janela do Container

"Quando alguém bater na porta 8080 do meu PC, mande para a porta 3000 do Container."

Onde estão meus containers?

Comando		O que ele faz?
	<code>docker ps</code>	"Quem está trabalhando agora?" (Lista ativos).
	<code>docker stop</code>	"Hora de descansar." (Para o container).
	<code>docker rm</code>	"Jogue a caixa fora." (Apaga o container).
	<code>docker logs</code>	"O que está acontecendo lá dentro?" (Vê mensagens).

Docker Network: O Walkie-Talkie

O Problema

Por padrão, um container é **surdo e mudo** para os outros. Eles vivem em mundos isolados e não conseguem conversar.

A Solução

Criar uma rede comum para eles. É como dar um **Walkie-Talkie** para cada container sintonizado na mesma frequência.



A Mágica: Se o container do banco se chama `meu-db`, o backend só precisa chamar por esse nome para conversar com ele. O **nome do container** vira o seu endereço!

Variáveis de Ambiente: O "Post-it"

O que é?

São informações que mudam conforme a necessidade, como a senha do banco de dados ou a porta que o app vai usar.

Pense nelas como **instruções dinâmicas** coladas na caixa do container.

Como passar?

Você usa a flag `-e` (de Environment) na hora de rodar o container:

```
docker run -e SENHA=123 meu-app
```



Por que usar? Para não escrever senhas direto no código! Isso traz **segurança** (não sobe pro GitHub) e **flexibilidade** (muda o comportamento sem precisar criar uma nova imagem).

Docker Compose: O Maestro

O Maestro da Orquestra

Rodar 5 containers na mão é chato e difícil. Você tem que lembrar de todas as portas, redes e nomes.

O **Docker Compose** é o maestro que coordena todos os músicos (containers) para tocarem juntos.

Você escreve tudo em um arquivo **YAML** e usa apenas um comando:

```
docker compose up
```

```
services:
  backend:
    build: ./api
    ports:
      - "8000:8000"

  db:
    image: postgres:16
    environment:
      - POSTGRES_PASSWORD=123
```

Arquivo .env: O Cofre

O que é?

Um arquivo escondido onde você guarda as senhas e segredos do seu projeto.

```
DB_PASSWORD=123456  
API_KEY=abc_xyz
```

Dica Prática

O Docker Compose lê esse arquivo sozinho e entrega as senhas para os containers sem você precisar digitar nada.



Regra de Ouro

Nunca, jamais, em hipótese alguma, mande o arquivo .env para o GitHub. Suas senhas seriam expostas para o mundo!

Volumes: A Memória Eterna

O Problema

Containers são temporários. Se você apaga o container do banco de dados, **todos os dados somem** para sempre!

A Solução

Volumes funcionam como um **HD Externo**. O container pode quebrar ou ser deletado, mas os dados continuam salvos no seu PC.

Como escrever no Docker Compose:

volumes:

- `./meus-dados:/var/lib/postgresql/data`

Resumo de Comandos (Para não esquecer)

AÇÃO	COMANDO	O QUE FAZ?
 Criar	<code>docker build -t nome .</code>	Cria a imagem (receita) do seu app.
 Rodar	<code>docker run -d -p 80:80 nome</code>	Inicia o container em segundo plano.
 Ver	<code>docker ps</code>	Lista todos os containers rodando agora.
 Compose	<code>docker compose up</code>	Sobe todos os serviços de uma vez só.
 Parar	<code>docker compose down</code>	Para e remove tudo o que o Compose subiu.

O Desafio de Hoje!

 **Projeto:** Frontend (Next.js) + Backend (Express) + Banco (PostgreSQL)

ETAPA	TAREFA PRÁTICA	TEMPO SUGERIDO
Etapa 1	Criar Dockerfile do Backend	🕒 30 min
Etapa 2	Subir PostgreSQL + Criar Rede	🕒 30 min
Etapa 3	Criar Dockerfile do Frontend	🕒 20 min
Etapa 4	Unir tudo no Docker Compose + .env	🕒 40 min